Alternate Ninja architecture structures

Putting the `NinjaAPI` object in urls.py and using `Router` instances to group API endpoints is just one approach. This tends to be how I organize my code, as it keeps the endpoint definitions in the same app as their model and view code. On occasion, I also separate out the schemas into their own file, creating a schemas.py to go along with the api.py file.

An alternate structure I've used in the past is to make the API its own Django app. In this case, you don't need the `Router` instance. Instead of calling `.add_router()` on the `NinjaAPI` object, you can use the instance of the `NinjaAPI` object directly: the HTTP method decorators are available on it as well. If you want to put all of your API code together in an app, you can create the `NinjaAPI` object in that app and import the instance into whatever files contain your endpoints.

This style of structure is common when using an alternate to Ninja called the Django REST Framework (as discussed later in the chapter). It is done this way, as the DRF requires a few more files to configure, and keeping it all in the same directory makes more sense.

11.3 Other API libraries

Ninja is definitely my preferred library for building APIs with Django. There are other choices out there, though. Predating Ninja is the *Django REST Framework* (DRF). The DRF (https://www.django-rest-framework.org/) is the granddaddy of Django REST API tools, and its first version was built before Python type hints existed. As such, there is a lot more code that needs to be written to build an API, code that Ninja does for you by understanding Python type hints.

The same basic ideas apply, though. Instead of functions, you typically use class methods, implementing a class for each object with methods corresponding to the GET, POST, and other HTTP actions. Classes also get used to declare serialization, and tools that map Django ORM objects are available. Like with Ninja, a multitude of authentication mechanisms are provided.

Like Ninja, the DRF comes with a web-based interface to interact with the API, which is very useful for testing. Unlike Ninja, the DRF supports several response-body formats. By using its serialization classes, changing from JSON to XML is a matter of a configuration parameter. DRF does this through the use of `Renderer` classes, and in fact, its web interface is built using this same technology. The DRF ships with renderers for JSON, HTML forms, Django templates, and more. There are also third-party add-ons for XML, YAML, three variations on JSON, MessagePack, Excel, CSV, Pandas, and Latex.

If your project needs payloads that are something other than JSON, then DRF is the way to go. This can also be used as a data export tool. For example, you could build an API with a single call whose output is YAML based on a group of nested serializers.

The DRF is definitely richer than Ninja, having more features and giving more control over how things work. The downside is it requires a lot more code, and for